Sistemas Operativos

Licenciatura em Engenharia Informática
Licenciatura em Engenharia Computacional
Licenciatura em Física (opcional)

Ano letivo 2025/2026

Pedro Azevedo Fernandes (paf@ua.pt)

Deadlocks

Preemptable and Nonpreemptable Resources

Tanenbaum

Sequence of events required to use a resource

1. Request the resource.
2. Use the resource.
3. Release the resource.

Resource Acquisition (1)

Tanenbaum

```
typedef int semaphore;
semaphore resource_1;


void process_A(void) {
        down(&resource_1);
        use_resource_1( );
        up(&resource_1);
}
```

(a)

```
typedef int semaphore;
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
        down(&resource_1);
        down(&resource_2);
        use_both_resources( );
        up(&resource_2);
        up(&resource_1);
}
```

(b)

Figure 6-1. Using a semaphore to protect resources.

(a) One resource. (b) Two resources.

.

Resource Acquisition (2)

Tanenbaum

```
typedef int semaphore;
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}

void process_B(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}
```

(a)

```
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}

void process_B(void) {
    down(&resource_2);
    down(&resource_1);
    use_both_resources( );
    up(&resource_1);
    up(&resource_2);
}
```

(b)

Figure 6-2. (a) Deadlock-free code. (b) Code with a potential deadlock.

Deadlock Definition

Tanenbaum

A set of processes is deadlocked if …

- Each process in the set waiting for an event
- That event can be caused only by another process

Conditions for Resource Deadlocks

Tanenbaum

Four conditions that must hold:

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait condition

Deadlock Modeling (1)

Tanenbaum

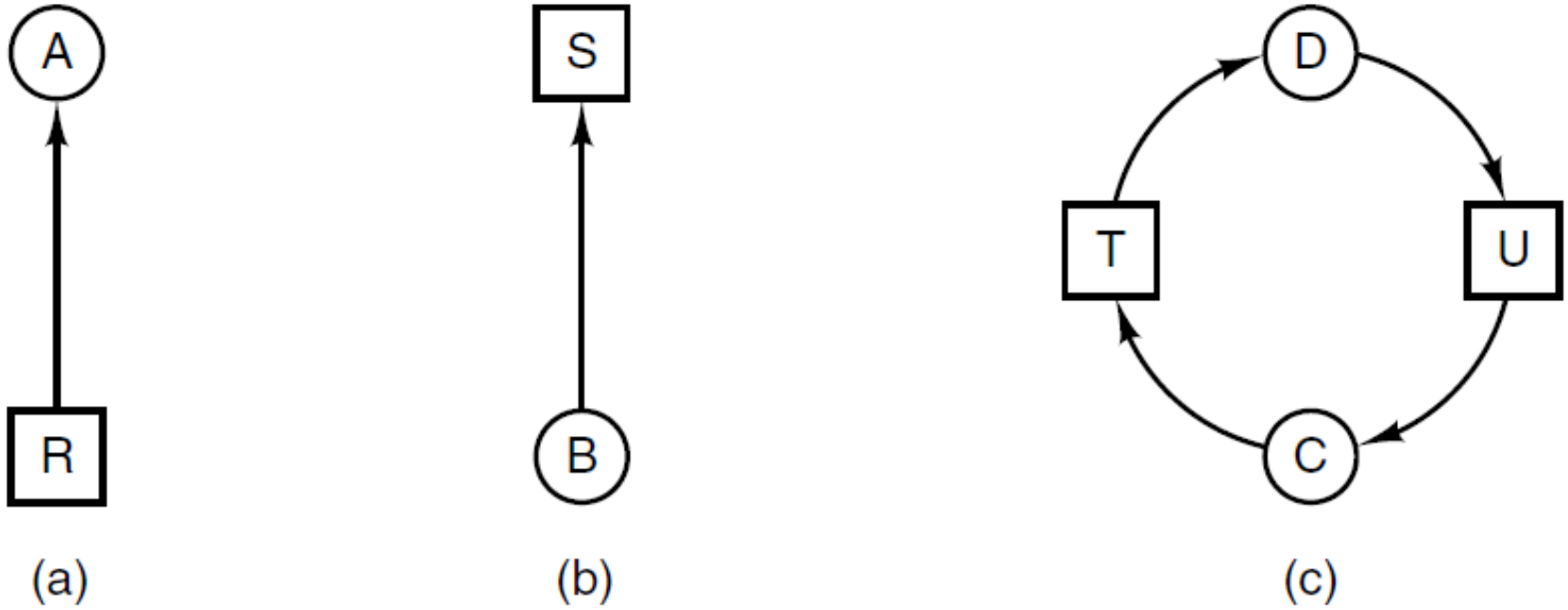

Figure 6-3. Resource allocation graphs. (a) Holding a resource. (b) Requesting a resource. (c) Deadlock.

Deadlock Modeling (2)

Tanenbaum

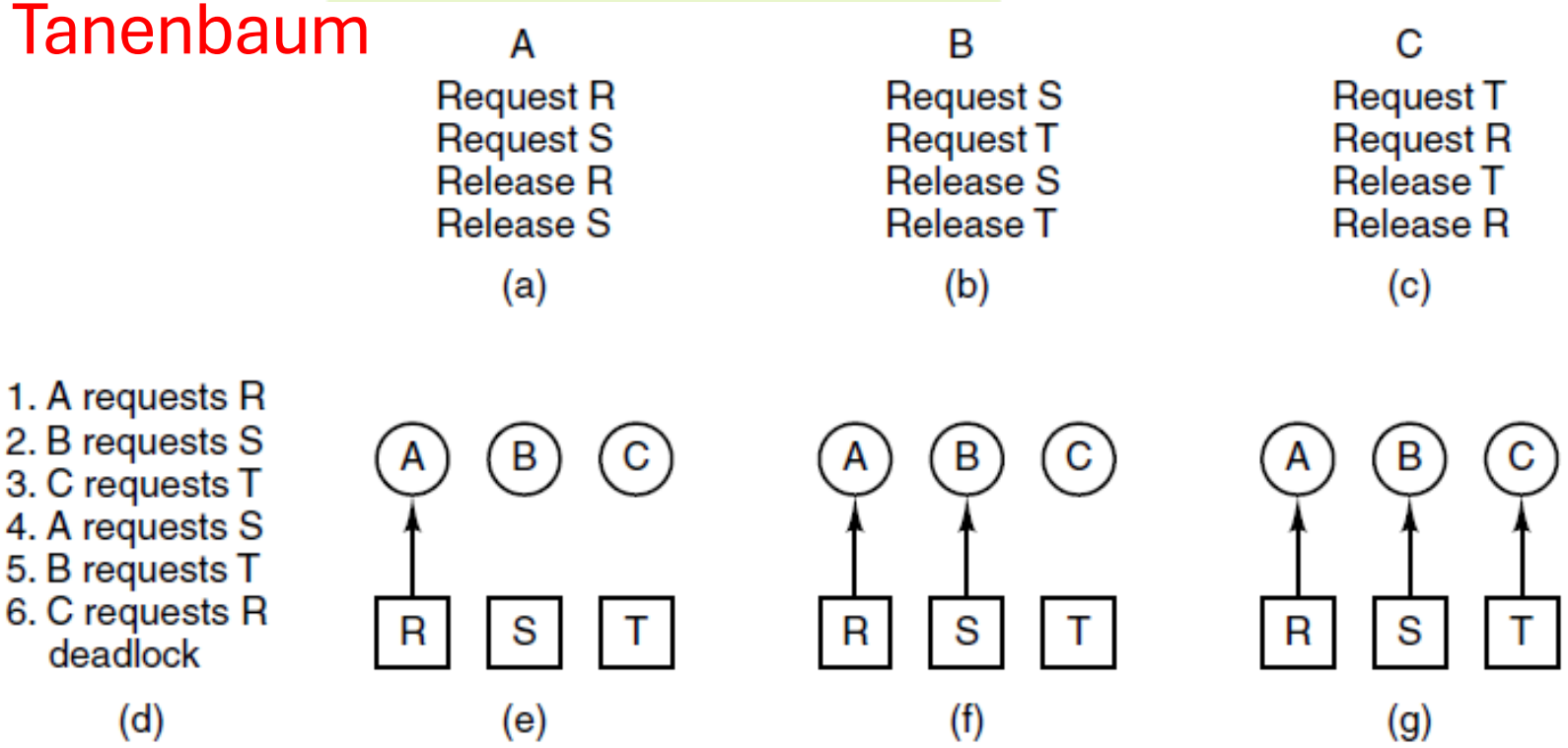

Figure 6-4. An example of how deadlock occurs and how it can be avoided.

Deadlock Modeling (3)

Tanenbaum

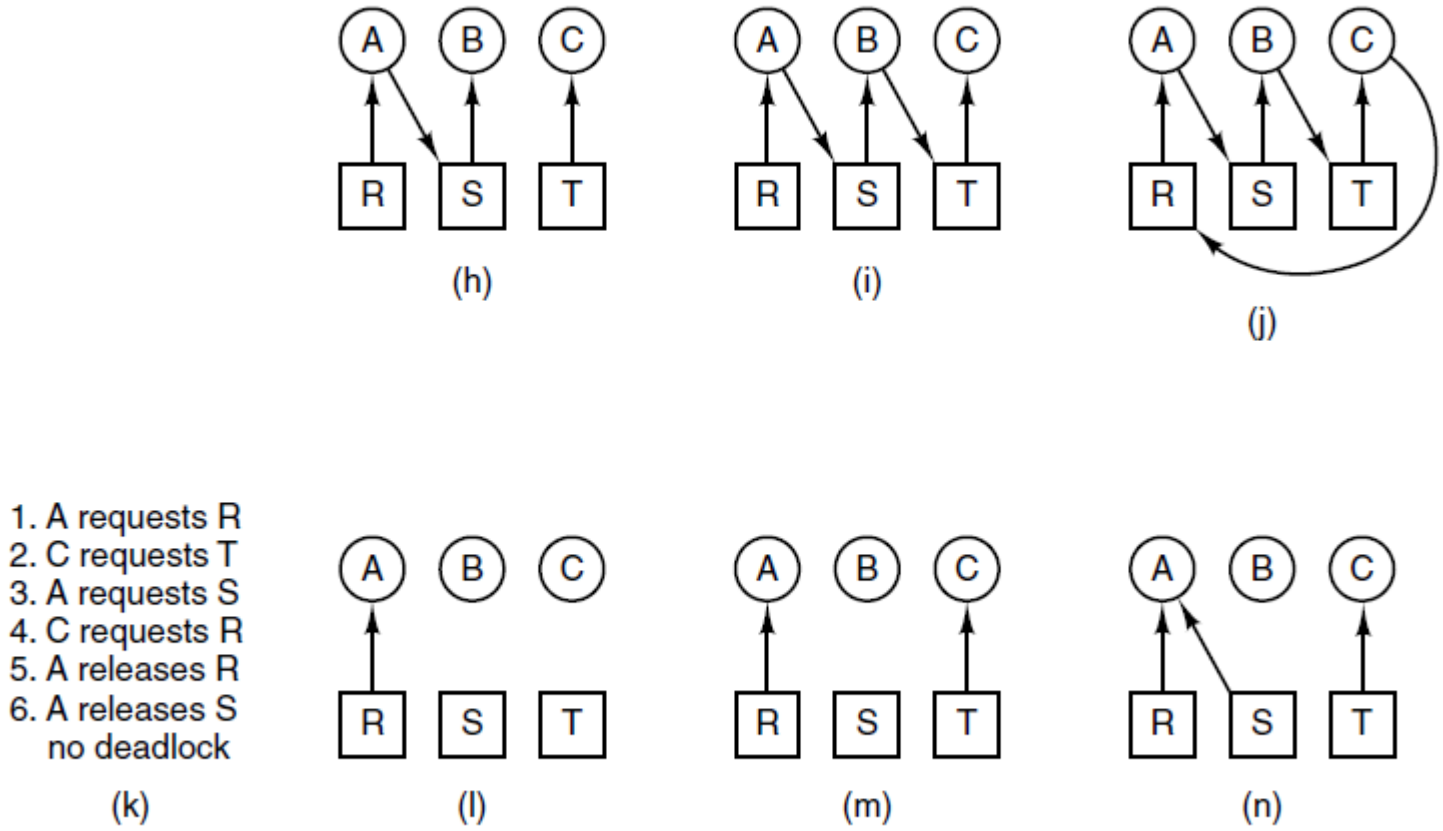

Figure 6-4. An example of how deadlock occurs and how it can be avoided.

Deadlock Modeling (4)

Tanenbaum

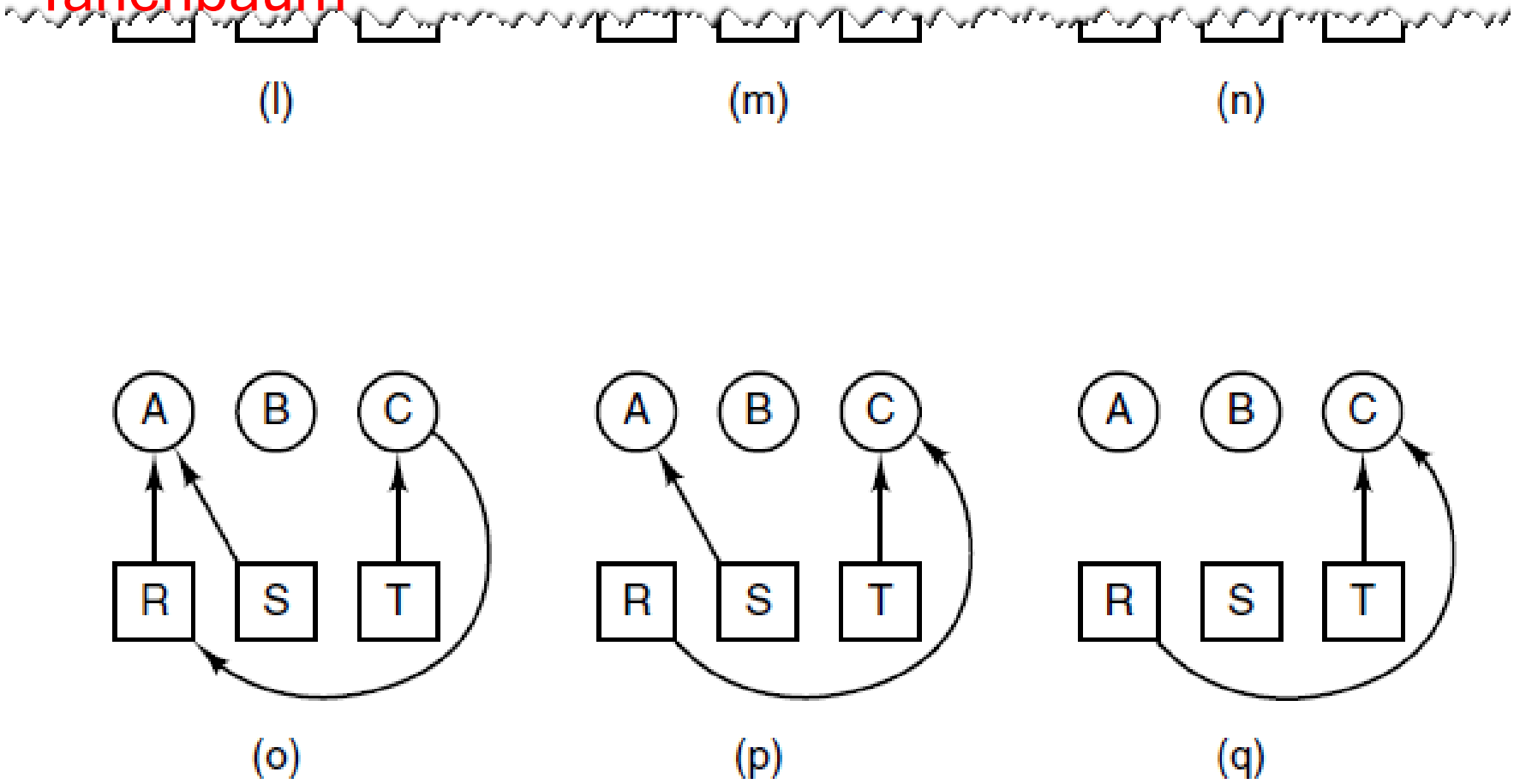

Figure 6-4. An example of how deadlock occurs and how it can be avoided.

Deadlock Modeling (5)

Tanenbaum

Strategies are used for dealing with deadlocks:

1. Ignore the problem, maybe it will go away.
2. Detection and recovery. Let deadlocks occur, detect them, and take action.
3. Dynamic avoidance by careful resource allocation.
4. Prevention, by structurally negating one of the four required conditions.

Safe States (1)

Gottlieb

The idea is to avoid deadlocks given some extra knowledge.

- Naturally, the resource manager knows how many units of each resource the system contains.
- The manager, which is the agent responsible for granting resources, also knows how many units of each resource it has given to each process and thus how many units remain available.

Safe States (2)

Gottlieb

- It would be great to see all the programs in advance and thus know all future requests as we did in the previous section, but that is asking for too much.
- Instead, when each process starts, it announces its maximum usage. That is, each process, *before* making any resource requests, must tell the resource manager the maximum number of units of each resource the process can possible need. This is called the **claim** of the process.

Safe States (3)

Gottlieb

- If the claim is greater than the total number of units in the system the resource manager kills the process when receiving the claim (or returns an error code so that the process can make a smaller claim).
- If during the run the process asks for more than its claim, the process is aborted (or an error code is returned and no resources are allocated).
- If a process claims more than it really needs, the result is that the resource manager will be more conservative than need be and there will likely be more waiting.

Safe States (4)

Gottlieb

- Examples of valid/invalid claims: Assume there is only one resource type, the system contains 5 units of that resource, and there are two processes P and Q.
- If P claims 7 units, that is invalid.
- If P and Q each claim 4 units, that is OK.
- If P claims 3 units, then requests 3 units, then the request is granted, and then P requests 3 more units, that last request is invalid.
- If P claims 4 units, then requests 3 units, then the request is granted, then P returns 2 units, then P requests 3 more, that is OK.

Safe States (5)

Gottlieb

Definition:

A state is **safe** if there is an ordering of the processes such that: if the processes are run in this order, they will all terminate (assuming none exceeds its claim, and assuming each would terminate if all its requests are granted).

Safe States (6)

Gottlieb

How the Resource Manager Determines if a State is Safe

- The manager has the following information available, which enable it to determine safety.
- Since the manager knows all the claims, it can determine the maximum amount of additional resources each process can request.
- The manager knows how many units of each resource it has left.

Safe States (7)

Gottlieb

- The manager then follows the following procedure, which is part of **Banker's Algorithms** discovered by Dijkstra, to determine if the state is safe.
- If there are no processes remaining, the state is **safe**.

Safe States (8)

Gottlieb

- Choose a process P whose maximum additional request for each resource type is less than or equal to what remains for that resource type.
  - If no such process can be found, then the state is **not safe**.
  - If such a P can be found, the banker (manager) knows that, if it refuses all requests except those from P, then it will be able to satisfy all of P's requests.

Safe States (9)

Gottlieb

- The banker now pretends that P has terminated (since the banker knows that it can guarantee this will happen). Hence the banker pretends that all of P's currently held resources are returned. This makes the banker richer and hence perhaps a process that was not eligible to be chosen as P previously, can now be chosen.
- Repeat these steps.

Safe States – Examples

Gottlieb

process	initial claim	current alloc	max add'l
X	3	1	2
Y	11	5	6
Z	19	10	9
Total		16	
Available		6	

A safe state with 22 units of one resource

Safe States – Examples

Gottlieb

process	initial claim	current alloc	max add'l
X	3	1	2
Y	11	5	6
Z	19	12	7
Total		18	
Available		4	

An unsafe state with 22 units of one resource

Banker's Algorithm (Dijkstra) for a Single Resource (1)

Gottlieb

- Before execution begins, ensure that the system is safe. That is, check that no process claims more than the manager has.
  - If the check fails, then the offending process is trying to claim more of some resource than exists in the system and hence cannot be guaranteed to complete even if run by itself.

Banker's Algorithm (Dijkstra) for a Single Resource (2)

Gottlieb

- When the manager receives a request, it **pretends** to grant it, and then checks if the resulting state is safe:
  - If it is safe, the request is really granted;
  - if it is not safe, the process is blocked (that is, the banker does not grant the request at this time).

Banker's Algorithm (Dijkstra) for a Single Resource (3)

Gottlieb

- When a resource is returned, the manager checks to see if the first pending request can be granted (i.e., if the result would now be safe).
  - If so, the first pending request is granted.
  - Whether or not the first pending request was granted, the manager checks to see if the next pending request can be granted, etc.